

📖 AI/ML Internship – Day 74 Notes & Tasks

🟢 Module 9: LangChain Fundamentals – Document Loaders, Text Splitters, Chains & Retrievers

📌 Part 1: Recap of Day 73

Yesterday we learned:

- ✅ What is a Retriever?
- ✅ Dense Retrieval
- ✅ Sparse Retrieval
- ✅ BM25
- ✅ Hybrid Search

Today we will learn:

- What is LangChain?
- Why LangChain is used
- Document Loaders
- Text Splitters
- Chains
- Retrievers in LangChain
- Building a simple LangChain RAG pipeline

📌 Part 2: What is LangChain?

Definition

LangChain is an open-source framework used to build applications powered by Large Language Models (LLMs).

It provides tools for:

- Loading documents
- Splitting text
- Creating embeddings

- Connecting vector databases
- Building retrievers
- Managing prompts
- Adding conversation memory

Simple Definition

LangChain is a toolkit for building AI applications.

Without LangChain, developers must manually write a large amount of code. LangChain simplifies the process.

📌 Part 3: Why Do We Use LangChain?

Suppose we want to build a PDF chatbot.

Without LangChain:

- Read PDF manually
- Split text manually
- Create embeddings manually
- Connect vector database manually
- Build retrieval logic manually

With LangChain:

```
loader = PyPDFLoader("file.pdf")
splitter = RecursiveCharacterTextSplitter()
vectorstore = Chroma.from_documents(...)
retriever = vectorstore.as_retriever()
```

Much less code is required.

📌 Part 4: Installing LangChain

Install the required libraries:

```
pip install langchain
```

```
pip install langchain-community
```

```
pip install langchain-openai
```

```
pip install chromadb
```

```
pip install sentence-transformers
```

```
pip install pypdf
```

📌 Part 5: Document Loaders

What is a Document Loader?

A Document Loader reads data from external files and converts it into a format that LangChain can process.

Supported file types:

- PDF
- TXT
- DOCX
- CSV
- HTML
- Web pages

Example: Loading a PDF

```
from langchain_community.document_loaders import PyPDFLoader
```

```
loader = PyPDFLoader("college.pdf")
```

```
documents = loader.load()
```

```
print(documents[0].page_content)
```

Output

Admission Fee: ₹25,000 Eligibility: Plus Two Duration: 3 Months

The PDF is converted into LangChain document objects.

📌 Part 6: Text Splitters

Why Split Text?

LLMs cannot efficiently process very large documents.

Example:

- 200-page PDF
- 50,000 words

We split the document into smaller chunks.

RecursiveCharacterTextSplitter

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
```

```
splitter = RecursiveCharacterTextSplitter(  
    chunk_size=500,  
    chunk_overlap=50  
)
```

```
chunks = splitter.split_documents(documents)
```

```
print(len(chunks))
```

Explanation

Parameter	Meaning
-----------	---------

chunk_size=500	Maximum characters per chunk
----------------	------------------------------

chunk_overlap=50	50 characters shared between chunks
------------------	-------------------------------------

✦ Part 7: Why Use Chunk Overlap?

Example:

Chunk 1

AI Internship starts on August 1.

Chunk 2

August 1 is the official joining date.

Without overlap, important context may be lost.

Overlap helps preserve information across chunks.

✦ Part 8: Creating Embeddings

```
from langchain.embeddings import HuggingFaceEmbeddings
```

```
embedding = HuggingFaceEmbeddings(  
    model_name="all-MiniLM-L6-v2"  
)
```

This converts text chunks into embeddings.

✦ Part 9: Storing in ChromaDB

```
from langchain.vectorstores import Chroma
```

```
vectorstore = Chroma.from_documents(  
    documents=chunks,  
    embedding=embedding  
)
```

Now all chunk embeddings are stored in the vector database.

✦ Part 10: Creating a Retriever

```
retriever = vectorstore.as_retriever(  
    search_kwargs={"k":3}  
)
```

This creates a retriever that returns the top 3 relevant chunks.

✦ Part 11: Retrieving Documents

```
docs = retriever.invoke(  
    "What is the internship duration?"  
)
```

for doc in docs:

```
    print(doc.page_content)
```

Output:

Internship Duration: 3 Months

📌 Part 12: What is a Chain?

Definition

A Chain connects multiple LangChain components into a workflow.

Example:

Question → Retriever → LLM → Answer

Simple Example

```
from langchain.chains import RetrievalQA
```

```
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=retriever
)
```

The chain automatically performs:

- Retrieval
- Context preparation
- LLM response generation

📌 Part 13: Complete LangChain Workflow

Upload PDF

Document Loader

Text Splitter

Embeddings

ChromaDB

Retriever

LLM

Final Answer

📌 Part 14: Real-World Applications

Company Knowledge Assistant

Search HR documents.

Legal Assistant

Search contracts.

Medical Assistant

Search treatment guidelines.

Research Assistant

Search research papers.

Educational Chatbot

Search textbooks and lecture notes.

📌 Part 15: Advantages of LangChain

- ☒ Reduces development time
- ☒ Modular architecture
- ☒ Easy integration with vector databases
- ☒ Supports conversation memory
- ☒ Works with many LLM providers

📌 Part 16: Limitations

- ☒ Can be complex for beginners
- ☒ Frequent version changes
- ☒ Some components may require additional configuration

📌 Part 17: Theory Questions

1. What is LangChain?

LangChain is an open-source framework for building applications powered by Large Language Models.

2. What is a Document Loader?

A Document Loader reads external files and converts them into LangChain document objects.

3. Why is text splitting required?

Text splitting divides large documents into smaller chunks that can be processed efficiently by LLMs.

4. What is a Retriever?

A Retriever searches the vector database and returns the most relevant document chunks.

5. What is a Chain?

A Chain connects multiple LangChain components into a single workflow.

Part 18: Interview Questions

Explain the role of LangChain in a RAG system.

LangChain connects document loading, chunking, embeddings, vector databases, retrievers, and LLMs into a unified workflow.

Why do we use RecursiveCharacterTextSplitter?

It splits documents into manageable chunks while preserving context through chunk overlap.

What is the purpose of `as_retriever()`?

It converts a vector store into a retriever that can perform similarity searches.

What is RetrievalQA?

RetrievalQA is a LangChain chain that combines document retrieval with LLM-based question answering.

Name three components of a LangChain RAG pipeline.

- Document Loader
- Text Splitter

- Retriever

Tasks for Day 74

Theory

- Define LangChain.
- Explain Document Loaders.
- Why do we use Text Splitters?
- What is a Retriever in LangChain?
- What is a Chain?

Practical Task 1

Install all required LangChain libraries.

Practical Task 2

Load a PDF using PyPDFLoader.

Practical Task 3

Split the PDF into chunks using RecursiveCharacterTextSplitter.

Practical Task 4

Store the chunks in ChromaDB and create a retriever.

Practical Task 5

Retrieve the Top 3 relevant chunks for five different questions.